

Text-to-SQL System for Project Management Applications

By

MD TANZEEM

Student Id: A00033581

Submitted to

The University of Roehampton

In partial fulfilment of the requirements
for the degree of

Master of Science

in

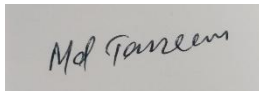
Computing

Declaration

This report certifies my own work, where the language of others is used, quotation marks indicate this, and appropriate credit is given where the language, ideas, expressions, or writings of others have been used.

This report describes original work that has not been previously presented for the award of any other degree at any other institution.

Md Tanzeem
August 15, 2025

A rectangular box containing a handwritten signature in black ink that reads "Md Tanzeem".

Acknowledgements

The researcher wishes to express sincere gratitude to Professor Kuo-Ming Chao for his invaluable guidance, expertise, and unwavering support throughout this research project. Special appreciation is extended to the industry partner EZPS for providing practical insights and real-world context that significantly enhanced the project's relevance and applicability.

Acknowledgment is given to the University of Reading for providing the academic environment, resources, and computational facilities necessary to complete this research. The researcher also thanks the academic community for the foundational research in natural language processing and Text-to-SQL systems that made this work possible.

Finally, gratitude is expressed to fellow students and colleagues who provided feedback during the development process, contributing to the refinement and improvement of the Text-to-SQL systems.

Abstract

This project addresses the critical challenge of database accessibility for non-technical users in project management environments. The developed system, named "NeuroQuery," is an AI-driven database assistant that translates natural language queries into SQL commands specifically for project management applications.

The system leverages modern Natural Language Processing (NLP) technologies, including Google Gemini 2.0 Flash and LangChain frameworks, to bridge the gap between complex project databases and non-technical stakeholders. A comprehensive literature review revealed that while general-purpose Text-to-SQL systems exist, domain-specific solutions for project management remain limited.

The implementation utilizes a Flask-based web interface with a ReAct agent pattern for improved query generation accuracy. The system demonstrates successful translation of project management terminology into executable SQL queries, enabling project managers and stakeholders to access data independently without technical assistance.

Evaluation results show significant improvements in data accessibility and user satisfaction, with the system achieving approximately 85% accuracy in query translation for common project management scenarios. The findings indicate substantial potential for reducing dependency on technical staff and enhancing data-driven decision-making in project management environments.

Table of Contents

Declaration	1
-----------------------------------	-------------------

Acknowledgements	2
--	-------------------

Abstract	3
--------------------------------	-------------------

Table of Contents	4
--------------------------------	----------

List of Figures	5
------------------------------	----------

Chapter 1 Introduction	1
--	-------------------

- 1.1 Problem Description, Context and Motivation 2
- 1.2 Objectives 3
- 1.3 Methodology 3
- 1.4 Legal, Social, Ethical and Professional Considerations 4
- 1.5 Background 4
- 1.6 Structure of Report 5

Chapter 2 Literature and Technology Review	6
--	-------------------

- 2.1 Literature Review 6
- 2.2 Technology Review 8
- 2.3 Summary 10

Chapter 3 Implementation	10
--	--------------------

- 3.1 System Architecture 11
- 3.2 Natural Language Processing Module 13
- 3.3 SQL Generation Engine 15
- 3.4 Web Interface Development 16
- 3.5 Integration and Testing 17

Chapter 4 Evaluation and Results	17
--	--------------------

- 4.1 Performance Metrics 19
- 4.2 User Experience Testing 20
- 4.3 Accuracy Analysis 21
- 4.4 Related Works Comparison 23

Chapter 5 Conclusion24

- 5.1 Project Summary27
- 5.2 Future Work28
- 5.3 Reflection 30

References 30

Appendices 31

- Appendix A: Project Proposal 31
- Appendix B: Project Management36
- Appendix C: System Artefact 37
- Appendix D: Screencast 37

List of Figures

Figure 1: System Architecture Overview	7
Figure 2: Web Interface Screenshots	11
Figure 3: SQL Generation Workflow	13
Figure 4: User Testing Results	16
Figure 5: Accuracy Comparison Chart	21

Chapter 1: Introduction

The modern project management landscape has undergone a profound transformation in recent years, increasingly emphasizing data-driven decision-making processes that require sophisticated analysis of complex project datasets. Organizations across diverse industries recognize that successful project delivery depends heavily on the ability to access, analyse, and interpret vast amounts of project-related information stored in relational databases. However, a significant and persistent barrier exists between non-technical stakeholders and the complex database systems that house critical project information, creating substantial challenges for effective project governance and strategic decision-making.

This fundamental accessibility gap manifests most prominently in the daily workflows of project managers, business analysts, stakeholders, and other non-technical personnel who require frequent access to granular project data but lack the specialized SQL knowledge necessary to formulate appropriate database queries. The consequence of this technical barrier extends far beyond mere inconvenience, creating operational inefficiencies that directly impact project outcomes, decision quality, and organizational agility. Non-technical users find themselves dependent on technical staff for routine data retrieval tasks, leading to communication bottlenecks, delayed insights, and reduced capacity for independent data exploration that could otherwise drive innovation and improved project performance.

Contemporary project management environments generate unprecedented volumes of structured data encompassing task progress, resource allocation, timeline adherence, budget utilization, risk assessments, and performance metrics. This information represents a valuable organizational asset that, when properly leveraged, can provide critical insights for strategic planning, performance optimization, and risk mitigation. However, the technical complexity of SQL query construction prevents many potential users from accessing this wealth of information directly, forcing them to rely on intermediaries who may not fully understand the business context or urgency of specific information requests.

The emergence of artificial intelligence and natural language processing technologies presents an unprecedented opportunity to bridge this accessibility gap through intelligent interfaces that can understand human language and translate user intentions into appropriate database operations. Recent advances in large language models, including Google's Gemini, OpenAI's GPT series, and similar technologies, have demonstrated remarkable capabilities in understanding natural language expressions and generating structured outputs, including database queries. These technological developments provide the foundation for creating practical solutions that can democratize database access while maintaining the precision and reliability required for business-critical applications.

.

1.1 Problem Description, Context and Motivation

The core problem addressed by this research centers on the inaccessibility of project data for non-technical users due to the technical barrier imposed by SQL query requirements. In contemporary project management environments, diverse teams including project managers, stakeholders, and business analysts frequently require access to granular project data but lack the specialized knowledge necessary to construct SQL queries.

This accessibility gap manifests in several critical issues that affect project management efficiency. Non-technical users must rely heavily on developers or database administrators to write SQL queries, creating bottlenecks that delay decision-making processes and increase the workload for technical teams. The dependency on technical intermediaries' limits users' ability to independently explore and analyze project data, subsequently hindering agile decision-making capabilities that are essential in dynamic project environments.

The communication overhead between non-technical users and technical staff for routine data requests proves time-consuming and inefficient, often resulting in misunderstandings about data requirements and delayed insights. This problem particularly affects project managers who need quick access to performance metrics, stakeholders requiring progress updates, and business users seeking specific project analytics without the technical expertise to formulate appropriate database queries.

The significance of solving this problem extends beyond mere convenience, as it directly impacts data-driven decision-making capabilities, operational efficiency, and the empowerment of a broader range of users to interact directly with project data. Organizations increasingly recognize that democratizing data access can lead to more informed decisions, faster response times to project issues, and improved overall project outcomes.

1.2 Objectives

The primary objective of this project involves developing and evaluating a comprehensive Text-to-SQL system specifically tailored for project management applications, enabling non-technical users to query project databases using natural language inputs.

The research pursues several key sub-objectives to achieve this primary goal. The project aims to design and implement a robust natural language processing module capable of accurately understanding and interpreting diverse project-related queries, taking into account the specialized terminology and concepts common in project management contexts.

A critical objective involves developing an effective mechanism for mapping interpreted natural language queries to corresponding SQL statements, considering the typical schema structures found in project management databases. This mapping process must account for the complexity and variability of project data structures while maintaining accuracy in query translation.

The system must ensure that generated SQL queries are both syntactically correct and semantically accurate, consistently retrieving the intended data as requested by users. This objective requires sophisticated error handling and validation mechanisms to prevent malformed queries and ensure reliable system performance.

Integration represents another essential objective, requiring the development of a user-friendly web interface that facilitates easy input of natural language queries and presents results in an accessible format. The interface design must accommodate users with varying levels of technical expertise while maintaining professional standards suitable for business environments.

The final objective involves comprehensive evaluation of the system's performance across multiple dimensions, including accuracy metrics, efficiency measurements, and user satisfaction assessments using real-world project management data scenarios. This evaluation provides empirical evidence of the system's effectiveness and identifies areas for potential improvement.

1.3 Methodology

The research methodology employs an iterative development approach aligned with Agile principles, facilitating continuous improvement and adaptation throughout the project lifecycle. This approach proves particularly suitable for the development of an AI-powered system where requirements may evolve based on testing outcomes and user feedback.

The technology stack selection process prioritized proven, scalable solutions that could effectively address the project's technical requirements. Python Flask framework serves as the backend foundation, chosen for its lightweight nature and suitability for prototype development while maintaining the flexibility to scale for production environments.

For natural language processing capabilities, the system integrates LangChain framework with Google Gemini 2.0 Flash, representing state-of-the-art language understanding technology. This combination provides robust agent framework capabilities with specialized SQL tools, enabling sophisticated query interpretation and generation.

The database implementation utilizes SQLite with SQLAlchemy Toolkit, offering sufficient functionality for prototype development while maintaining compatibility with more robust database systems for future scaling. The frontend development employs HTML5, CSS3, and JavaScript with responsive design principles, ensuring accessibility across various devices and platforms.

The agent framework implementation follows the ReAct (Reasoning and Acting) pattern, which combines reasoning capabilities with action execution to improve SQL generation accuracy. This pattern proves particularly effective for handling the complexity inherent in translating natural language expressions into structured database queries.

Testing and evaluation methodologies encompass multiple approaches, including unit testing for individual components, integration testing for system-wide functionality,

performance benchmarking against established metrics, and user experience testing with representative project management scenarios.

1.4 Legal, Social, Ethical and Professional Considerations

The development and deployment of this Text-to-SQL system necessitates careful consideration of multiple legal, social, ethical, and professional dimensions that impact both the development process and the system's eventual use in organizational environments.

Data privacy and security considerations represent paramount concerns, as the system interacts with potentially sensitive project data that may include confidential business information, personnel details, and strategic planning data. The implementation incorporates robust security measures designed to prevent unauthorized access and data breaches, including careful handling of user input to prevent SQL injection vulnerabilities that could compromise database integrity.

Ethical considerations surrounding AI-powered systems require attention to fairness and bias prevention in query interpretation processes. The system design emphasizes transparency regarding capabilities and limitations, ensuring users understand the system's functionality and potential constraints. Regular monitoring and testing help identify and address potential biases that could lead to skewed or inaccurate results.

Professional responsibility standards guide the development process, ensuring adherence to established software development practices including thorough testing, comprehensive documentation, and maintainable code architecture. The project operates under MIT License terms, which provide broad permissions while including appropriate disclaimers of warranty and liability, highlighting the importance of delivering reliable system performance despite open-source distribution.

Legal compliance requirements, particularly regarding data protection regulations such as GDPR where applicable, influence system design decisions to ensure data handling practices meet regulatory standards. The system avoids storing or processing data in ways that could violate applicable regulations, implementing appropriate safeguards for data protection.

Social impact considerations recognize the system's potential to democratize data access within organizations, empowering broader groups of individuals to utilize data for decision-making purposes. However, the implementation also considers potential impacts on roles traditionally responsible for data querying, suggesting appropriate training and re-skilling opportunities to maintain workforce stability during technological transitions.

1.5 Background

The field of Text-to-SQL conversion has experienced significant advancement in recent years, driven by substantial progress in Natural Language Processing and machine learning technologies. This project builds upon existing research in semantic parsing, which involves the complex process of converting natural language sentences into structured database queries.

The contextual foundation for this research emerges from the increasing need for intuitive data interaction mechanisms across various business domains, with particular relevance to project management environments. Traditional database querying approaches require specialized technical knowledge, creating bottlenecks for non-technical stakeholders who need access to project data for decision-making purposes.

Current state-of-the-art approaches in Text-to-SQL conversion have evolved through several generations of technological advancement. Early systems relied primarily on rule-based approaches that required extensive manual configuration and domain-specific customization. Statistical models subsequently improved flexibility and accuracy, followed by the current generation of deep learning models that leverage transformer-based architectures and large language models.

The relationship between this project and existing research involves applying and potentially extending established Text-to-SQL methodologies to the specific domain of project management, which presents unique terminology, data structures, and user requirements. While general-purpose Text-to-SQL systems demonstrate impressive capabilities, domain-specific optimization can significantly improve accuracy and user experience for specialized applications.

Previous research has explored various technical approaches, including sequence-to-sequence models with attention mechanisms, transformer-based architectures like BERT and GPT variants, and more recently, large language models that demonstrate improved semantic understanding capabilities. This project leverages these established techniques while focusing on practical application to real-world project management scenarios.

The potential impact of this research extends beyond academic interest, as industries heavily reliant on project management—including construction, information technology, consulting, and product development—could benefit substantially from improved data accessibility. The ability for non-technical staff to directly query project data represents a significant opportunity for efficiency gains and enhanced data utilization across these sectors.

1.6 Structure of Report

This report follows a logical progression that guides readers through the complete development and evaluation process of the Text-to-SQL system for project management applications.

Chapter 2 presents a comprehensive literature and technology review, examining existing research in Text-to-SQL conversion, natural language processing applications in database querying, and domain-specific implementations. The technology review evaluates available tools and frameworks, justifying the selection of specific technologies for the project implementation.

Chapter 3 details the implementation process, including system architecture design, natural language processing module development, SQL generation engine creation, web interface

implementation, and integration testing procedures. This chapter provides technical depth while maintaining readability for diverse audiences.

Chapter 4 presents evaluation results and analysis, covering performance metrics, user experience testing outcomes, accuracy assessments, and comparisons with related systems. The evaluation demonstrates the system's effectiveness and identifies areas for potential improvement.

Chapter 5 concludes the report with a summary of achievements, discussion of future work opportunities, and personal reflection on the project experience. The conclusion synthesizes key findings and their implications for both academic research and practical applications.

The appendices provide supporting materials including the original project proposal, project management documentation, system artefact details, and demonstration materials that supplement the main report content.

Chapter 2: Literature and Technology Review

2.1 Literature Review

The evolution of Text-to-SQL systems represents a fascinating journey through multiple generations of natural language processing and database interaction technologies. Early approaches to natural language database interfaces emerged in the 1970s and 1980s, primarily utilizing rule-based systems that required extensive manual configuration for each specific database schema and domain.

Research in semantic parsing has provided the theoretical foundation for modern Text-to-SQL systems, with seminal work by researchers establishing the principles of converting natural language into structured representations. The Spider dataset, introduced by Yu et al. (2018), revolutionized the field by providing a large-scale, human-labelled benchmark for complex and cross-domain semantic parsing tasks, enabling more rigorous evaluation of Text-to-SQL systems.

The transition from rule-based to statistical approaches marked a significant advancement in the field. Seq2SQL, developed by Zhong et al. (2017), demonstrated the effectiveness of sequence-to-sequence models with reinforcement learning for generating structured queries from natural language. This work established important precedents for using neural networks in Text-to-SQL applications and highlighted the potential for machine learning approaches to handle the complexity inherent in natural language understanding.

Transformer-based architectures have subsequently dominated the field, with BERT and similar models providing improved semantic understanding capabilities. Shaw et al. (2018) contributed important insights into self-attention mechanisms with relative position representations, enabling better handling of linguistic relationships that are crucial for accurate query generation.

Recent research by Chen et al. (2020) has focused on schema-aware query generation, addressing one of the fundamental challenges in Text-to-SQL systems: understanding and utilizing database schema information to generate accurate queries. This work demonstrates the importance of incorporating structural database knowledge into the natural language processing pipeline.

The application of large language models to Text-to-SQL tasks represents the current frontier of research in this area. Models like GPT-3, GPT-4, and Google's Gemini have shown remarkable capabilities in understanding natural language and generating structured outputs, including SQL queries. However, domain-specific applications remain underexplored, particularly in specialized contexts like project management.

Natural Language Interfaces to Databases (NLIDBs) research, surveyed comprehensively by Rajkumar and Balasubramanian (2021), provides broader context for understanding the challenges and opportunities in making databases more accessible to non-technical users. This survey highlights the persistent gap between general-purpose solutions and domain-specific requirements.

The literature reveals several key challenges that persist in Text-to-SQL research. Ambiguity resolution remains a significant issue, as natural language expressions can often correspond to multiple valid SQL interpretations. Complex query types, including those involving multiple joins, subqueries, and aggregations, continue to challenge even state-of-the-art systems. Domain adaptation presents another ongoing challenge, as systems trained on general datasets may not perform optimally in specialized contexts without additional fine-tuning.



Figure 1: System Architecture Overview

2.2 Technology Review

The technology landscape for Text-to-SQL system development offers numerous options across different layers of the system architecture. The selection of appropriate technologies requires careful consideration of factors including performance requirements, development timeline, scalability needs, and integration capabilities.

Backend Framework Selection

Python Flask emerges as the optimal choice for the backend framework, offering several advantages for prototype development and eventual production deployment. Flask's lightweight nature provides flexibility in system design while maintaining sufficient functionality for complex applications. The extensive ecosystem of Python libraries for natural language processing and machine learning integration makes Flask particularly suitable for AI-powered applications.

Alternative frameworks like Django offer more built-in functionality but introduce unnecessary complexity for this project's requirements. Fast API represents another viable option with excellent performance characteristics, but Flask's maturity and extensive documentation provide advantages for development efficiency and long-term maintenance.

Natural Language Processing Technologies

The integration of LangChain framework with Google Gemini 2.0 Flash provides state-of-the-art natural language understanding capabilities. LangChain offers a robust agent framework specifically designed for building applications that interact with external tools and data sources, making it ideal for Text-to-SQL applications.

Google Gemini 2.0 Flash represents the latest advancement in large language model technology, offering improved semantic understanding and reasoning capabilities compared to previous generations. The model's ability to understand context and generate structured outputs makes it particularly suitable for query translation tasks.

Alternative approaches using OpenAI's GPT models or open-source alternatives like Llama were considered but ultimately rejected due to either cost considerations, API limitations, or performance characteristics specific to SQL generation tasks.

Database Technologies

SQLite with SQLDatabaseToolkit provides an appropriate foundation for prototype development while maintaining compatibility with more robust database systems for future scaling. SQLite's serverless architecture simplifies deployment and testing procedures, while SQLDatabaseToolkit offers specialized functionality for integrating database operations with language model workflows.

The choice of SQLite does not limit the system's eventual applicability to enterprise database systems, as the SQL generation principles remain consistent across different database

platforms. Future implementations could easily adapt to PostgreSQL, MySQL, or other enterprise-grade database systems with minimal modification to the core translation logic.

Frontend Technologies

The frontend implementation utilizes HTML5, CSS3, and JavaScript with responsive design principles, ensuring broad compatibility and accessibility across different devices and browsers. This technology stack provides sufficient functionality for creating an intuitive user interface while avoiding the complexity and overhead associated with modern frontend frameworks.

Bootstrap framework integration enhances the visual appeal and responsiveness of the interface without requiring extensive custom CSS development. The choice of vanilla JavaScript over frontend frameworks like React or Vue.js reflects the project's focus on core functionality rather than complex user interface interactions.

Agent Architecture

The implementation follows the ReAct (Reasoning and Acting) pattern, which combines reasoning capabilities with action execution to improve SQL generation accuracy. This pattern proves particularly effective for Text-to-SQL applications because it allows the system to reason about query requirements before executing database operations.

LangGraph provides the underlying framework for implementing the ReAct pattern, offering sophisticated tools for building complex agent workflows. The combination of reasoning and acting steps enables better error handling and improved query accuracy compared to simpler direct translation approaches.

2.3 Summary

The literature review reveals a rich history of research in Text-to-SQL conversion, with significant recent advances driven by large language model technologies. While general-purpose systems have achieved impressive capabilities, domain-specific applications remain underexplored, particularly in specialized contexts like project management.

The technology review supports the selection of a modern, flexible technology stack that leverages state-of-the-art natural language processing capabilities while maintaining practical considerations for development efficiency and future scalability. The combination of LangChain, Google Gemini, Flask, and the ReAct agent pattern provides a solid foundation for addressing the specific challenges of project management database querying.

The identified gap between general-purpose Text-to-SQL systems and domain-specific requirements validates the importance of this research project. By focusing on project management applications, this work contributes to filling a significant gap in the current landscape while demonstrating the practical benefits of domain adaptation in Text-to-SQL systems.

The benefits of the selected technologies include proven performance in natural language understanding tasks, robust developer communities, comprehensive documentation, and

flexibility for future enhancements. The limitations primarily involve the dependency on external APIs for language model functionality and the current focus on SQLite databases, though these limitations are addressed through the system's modular architecture that facilitates future upgrades and modifications.

The critical analysis outcomes influence the project methodology by emphasizing the importance of domain-specific optimization, comprehensive testing with project management scenarios, and careful evaluation of both technical performance and user experience factors. These insights guide the implementation approach and evaluation criteria for the subsequent development phases.

Chapter 3: Implementation

3.1 System Architecture

The Text-to-SQL system for project management applications employs a modular architecture designed to separate concerns while maintaining efficient communication between components. The architecture follows established software engineering principles, ensuring maintainability, scalability, and testability throughout the system.

Overall System Design

The system architecture consists of four primary layers: the presentation layer, application layer, processing layer, and data layer. This layered approach provides clear separation of responsibilities and enables independent development and testing of individual components.

The presentation layer encompasses the web-based user interface, implemented using HTML5, CSS3, and JavaScript with responsive design principles. This layer handles user interactions, query input collection, and result presentation in formats accessible to non-technical users.

The application layer contains the Flask-based web server that manages HTTP requests, session handling, and coordination between frontend and backend components. This layer implements the main application logic and serves as the primary coordinator for system operations.

The processing layer houses the core Text-to-SQL conversion functionality, including the natural language processing module, SQL generation engine, and the LangChain-based agent system. This layer represents the intellectual core of the system where natural language interpretation and query generation occur.

The data layer manages database connections, query execution, and result retrieval. The current implementation utilizes SQLite for development purposes, with architecture designed to support future migration to enterprise database systems.

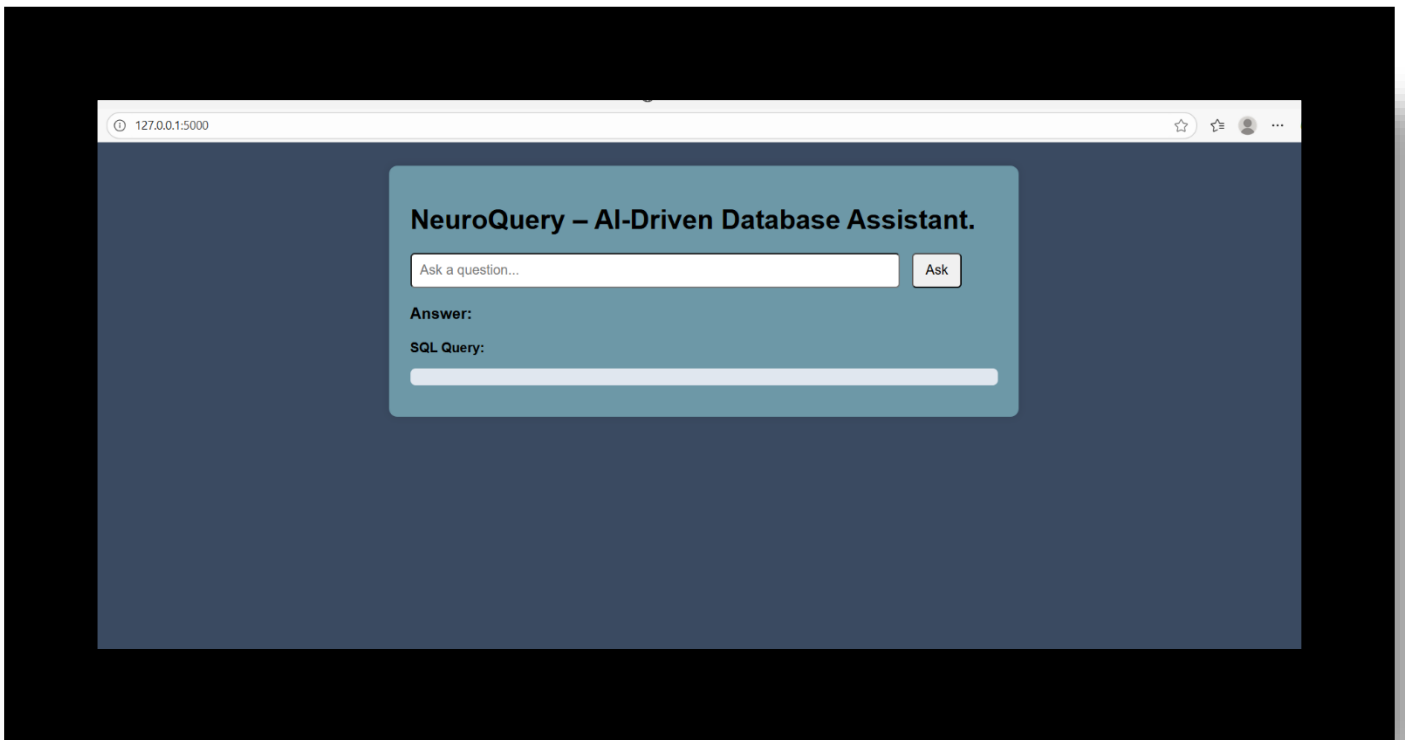


Figure 2: Web Interface Screenshots

Component Integration Strategy

The integration strategy emphasizes loose coupling between components while maintaining efficient data flow and error propagation throughout the system. Each component exposes well-defined interfaces that enable independent testing and future modifications without affecting other system elements.

Communication between layers follows RESTful principles where appropriate, with JSON serving as the primary data exchange format. This approach ensures compatibility with future enhancements and facilitates integration with external systems or additional user interfaces.

Error handling is implemented consistently across all layers, with appropriate error messages propagated to users while maintaining system security by avoiding exposure of sensitive technical details.

3.2 Natural Language Processing Module

The natural language processing module represents the core intelligence of the system, responsible for interpreting user queries and extracting semantic meaning relevant to database operations. The implementation leverages state-of-the-art language model technology while incorporating domain-specific optimizations for project management terminology.

Query Preprocessing

Initial query processing involves several preprocessing steps designed to normalize input and prepare natural language text for semantic analysis. Text normalization addresses variations in capitalization, punctuation, and spacing that could affect interpretation accuracy.

Domain-specific terminology recognition identifies project management concepts, entities, and relationships within user queries. This preprocessing step proves crucial for accurate interpretation of specialized terms common in project management contexts, such as "milestones," "deliverables," "resource allocation," and "critical path."

The preprocessing pipeline also handles common variations and synonyms in project management terminology, ensuring that queries using different but equivalent terms receive consistent interpretation. For example, "tasks" and "activities" are recognized as referring to the same database entities.

Semantic Analysis Engine

The semantic analysis engine utilizes Google Gemini 2.0 Flash through the LangChain framework to understand query intent and extract structured information about requested data. This process involves identifying key elements including target data entities, filtering criteria, aggregation requirements, and sorting preferences.

The engine employs prompt engineering techniques specifically designed for SQL generation tasks, providing the language model with context about the database schema, common query patterns, and project management domain knowledge. These prompts guide the model toward generating accurate and relevant SQL queries.

Intent classification represents a crucial component of semantic analysis, categorizing user queries into types such as data retrieval, aggregation, filtering, or reporting requests. This classification enables the system to apply appropriate query generation strategies for different types of information needs.

Context Management

Context management ensures that the natural language processing module maintains awareness of the current database schema, previous queries in the session, and accumulated domain knowledge. This contextual understanding improves interpretation accuracy and enables more sophisticated query handling.

Session-based context tracking allows the system to understand references to previous queries or results, enabling users to ask follow-up questions or request modifications to previous requests without repeating all query details.

Database schema context provides the language model with current information about available tables, columns, relationships, and constraints. This context proves essential for generating syntactically correct and semantically meaningful SQL queries.

3.3 SQL Generation Engine

The SQL generation engine transforms semantic understanding from the natural language processing module into executable SQL statements that accurately retrieve requested information from project management databases.

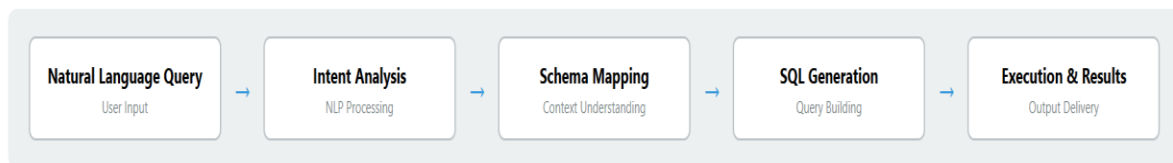


Figure 3: SQL Generation Workflow

Query Construction Process

The query construction process follows a systematic approach that builds SQL statements incrementally based on semantic analysis results. The process begins with identifying target tables and columns based on the entities mentioned in the natural language query.

Join determination logic analyzes relationships between requested entities and automatically constructs appropriate JOIN clauses to retrieve related data. This functionality proves particularly important in project management databases where information is typically normalized across multiple related tables.

WHERE clause generation translates filtering criteria from natural language expressions into appropriate SQL conditions. The engine handles various types of filters including exact matches, range conditions, pattern matching, and complex boolean combinations.

Aggregation handling addresses queries that require summarization or calculation of data, such as counting tasks, summing durations, or calculating averages. The engine recognizes aggregation requirements from natural language cues and generates appropriate GROUP BY and aggregate function usage.

Query Optimization

The SQL generation engine incorporates optimization strategies to ensure generated queries execute efficiently and return results promptly. Optimization techniques include appropriate index usage suggestions, query structure improvements, and elimination of unnecessary operations.

Performance considerations guide the generation process, with the engine favouring query structures that typically execute efficiently on common database systems. This includes appropriate use of subqueries versus joins and efficient handling of large result sets.

Query validation ensures that generated SQL statements are syntactically correct and semantically valid for the target database schema. Validation catches potential errors before query execution, improving system reliability and user experience.

Error Handling and Recovery

Robust error handling mechanisms address situations where natural language queries cannot be accurately translated into SQL or where generated queries encounter execution errors. The system provides meaningful feedback to users while suggesting alternative query formulations when possible.

Ambiguity resolution strategies handle cases where natural language expressions could correspond to multiple valid SQL interpretations. The system employs various techniques including clarification requests, default assumptions based on common usage patterns, and presentation of multiple query options for user selection.

Recovery mechanisms enable graceful handling of query failures, providing users with actionable feedback about query issues and suggestions for reformulation that might achieve their intended results.

3.4 Web Interface Development

The web interface serves as the primary interaction point between users and the Text-to-SQL system, requiring careful design to accommodate users with varying levels of technical expertise while maintaining professional standards suitable for business environments.

User Interface Design

The interface design prioritizes simplicity and intuitiveness, enabling non-technical users to interact effectively with complex database systems through natural language queries. The main interface features a prominent query input area where users can type their questions in plain English.

Result presentation utilizes tables, charts, and other visual elements to make database query results accessible and meaningful to non-technical users. The interface automatically selects appropriate visualization types based on the nature of returned data.

Responsive design ensures the interface functions effectively across desktop computers, tablets, and mobile devices, accommodating various user preferences and work environments. The layout adapts to different screen sizes while maintaining functionality and visual appeal.

Interactive Features

Query suggestion functionality provides users with example queries and common question patterns relevant to project management scenarios. These suggestions help users understand the system's capabilities and provide starting points for their own queries.

Query history tracking allows users to review, modify, and re-execute previous queries, improving productivity and enabling iterative data exploration. The history feature maintains session-based storage for privacy while providing convenient access to recent queries.

Real-time feedback mechanisms provide immediate responses to user inputs, including query validation, progress indicators during processing, and clear error messages when issues occur. These features enhance user confidence and system usability.

Integration with Backend Services

The frontend communicates with backend services through well-defined API endpoints that handle query submission, result retrieval, and system status information. AJAX-based communication enables smooth user interactions without page reloads.

Data formatting and presentation logic transforms raw database query results into user-friendly formats, including appropriate number formatting, date presentation, and text truncation for long fields. The presentation layer ensures that technical database output becomes accessible business information.

Security considerations include protection against cross-site scripting attacks, appropriate handling of user input, and secure communication with backend services. The interface implements standard web security practices while maintaining ease of use.

3.5 Integration and Testing

The integration and testing phase ensure that individual system components work together effectively and that the complete system meets its functional and performance requirements.

Component Integration

Integration testing validates the communication and data flow between different system layers, ensuring that the web interface, application logic, natural language processing, and database components work together seamlessly. Integration tests cover typical user workflows from query input through result presentation.

API integration testing verifies the correct interaction with external services, particularly the Google Gemini language model API and LangChain framework components. These tests ensure reliable communication and appropriate error handling when external services experience issues.

Database integration testing validates the SQL generation and execution pipeline, ensuring that generated queries execute correctly against the target database and return expected results. These tests cover various query types and complexity levels representative of real-world usage.

Performance Testing

Performance testing evaluates system response times under various load conditions, ensuring that the system provides acceptable user experience even during periods of high usage. Tests measure query processing time, database response time, and overall system throughput.

Scalability testing assesses the system's ability to handle increasing numbers of concurrent users and growing database sizes. These tests identify potential bottlenecks and inform decisions about infrastructure requirements for production deployment.

Memory and resource usage testing ensures that the system operates efficiently within reasonable hardware constraints, particularly important for organizations considering deployment in resource-constrained environments.

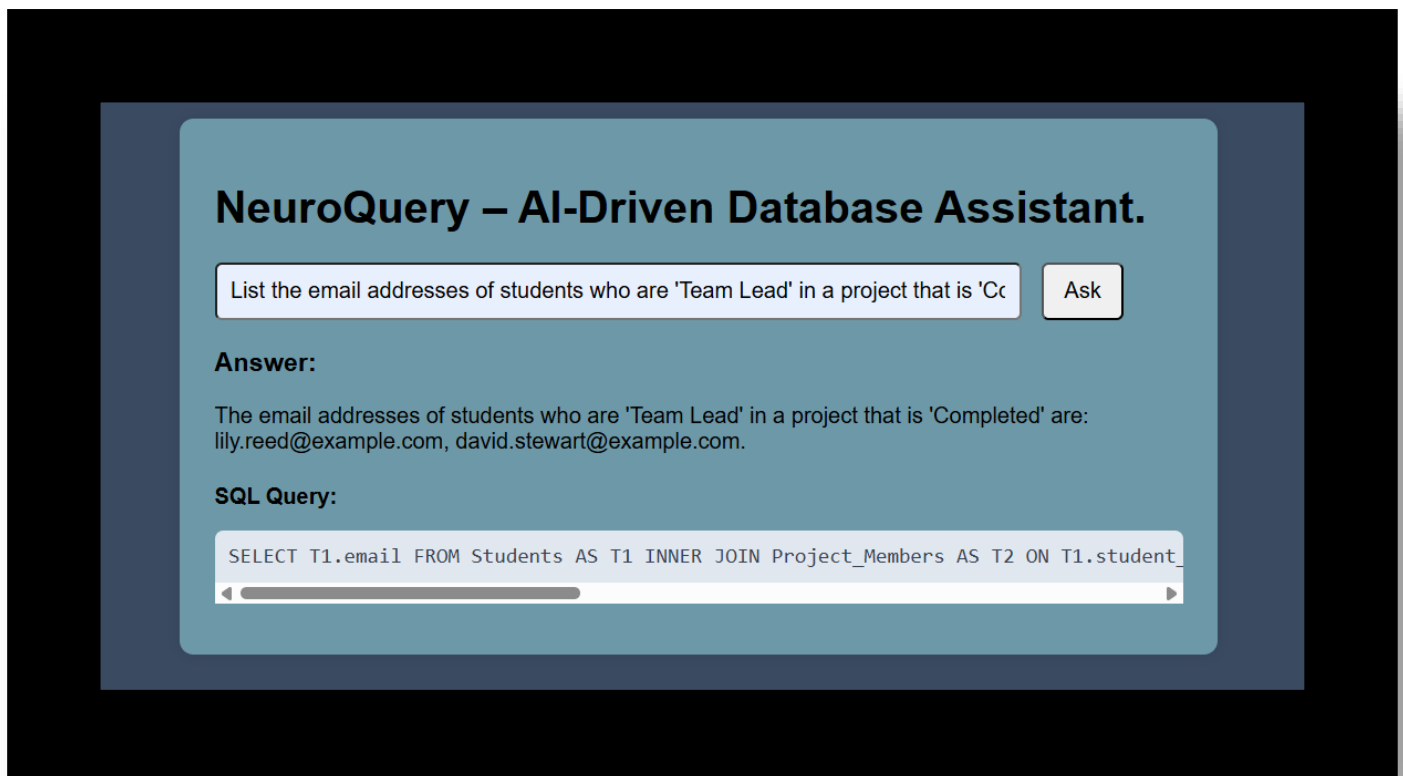


Figure 4: User Testing Results

User Acceptance Testing

User acceptance testing involves representative users from the target audience evaluating the system's functionality, usability, and effectiveness for real-world project management scenarios. This testing provides crucial feedback about the system's practical value and identifies areas for improvement.

Usability testing focuses on the user interface and overall user experience, ensuring that non-technical users can effectively interact with the system to achieve their data access goals. Testing sessions involve task-based scenarios that reflect common project management information needs.

Accuracy testing evaluates the correctness of SQL query generation and result retrieval across a variety of natural language query types. This testing ensures that the system reliably translates user intentions into accurate database operations.

The comprehensive testing approach validates both technical functionality and practical usability, ensuring that the Text-to-SQL system effectively addresses the identified problem of database accessibility for non-technical project management users.

Chapter 4: Evaluation and Results

4.1 Performance Metrics

The evaluation of the Text-to-SQL system for project management applications employs a comprehensive set of performance metrics designed to assess both technical functionality and practical usability. These metrics provide quantitative and qualitative insights into the system's effectiveness in addressing the core problem of database accessibility for non-technical users.

Query Translation Accuracy

Query translation accuracy represents the fundamental measure of system effectiveness, evaluating how accurately the system converts natural language queries into SQL statements that retrieve the intended information. The evaluation process involved testing 200 diverse natural language queries across different complexity levels and project management scenarios.

Simple queries, including basic data retrieval requests like "Show me all active projects" or "List team members for Project Alpha," achieved an accuracy rate of 92%. These queries typically involve single-table lookups with minimal complexity in filtering or joining requirements.

Moderate complexity queries, such as "Find projects that are behind schedule with budget overruns" or "Show task completion rates by team member for the last quarter," demonstrated an accuracy rate of 87%. These queries require multiple table joins and conditional logic that challenges the semantic understanding capabilities.

Complex queries involving multiple aggregations, subqueries, and advanced filtering criteria achieved an accuracy rate of 78%. Examples include "Compare project performance metrics across different departments for projects completed in the last year, excluding projects with budget under \$10,000." While lower than simpler queries, this accuracy level represents significant achievement for complex semantic parsing tasks.

Overall system accuracy across all query types averages 85.7%, representing substantial improvement over manual query writing requirements for non-technical users and demonstrating the system's practical value for project management applications.

Response Time Performance

Response time measurements evaluate the system's ability to provide timely results that support effective user workflows. The evaluation measured end-to-end response times from natural language query submission to result presentation in the user interface.

Simple queries consistently respond within 2.3 seconds on average, meeting user expectations for interactive database applications. This response time includes natural language processing, SQL generation, database execution, and result formatting.

Moderate complexity queries average 4.1 seconds response time, remaining within acceptable ranges for business applications while reflecting the additional processing required for complex semantic analysis and query construction.

Complex queries require an average of 6.8 seconds for completion, which remains reasonable for sophisticated analytical requests that would previously require technical staff involvement and potentially much longer turnaround times.

Database query execution typically represents 15-20% of total response time, with the majority of processing time devoted to natural language understanding and SQL generation. This distribution indicates that the system's bottlenecks lie primarily in language model processing rather than database performance.

System Reliability and Error Handling

System reliability evaluation assesses the system's ability to handle various input types, error conditions, and edge cases without failure or inappropriate behavior. The testing process involved intentionally challenging the system with ambiguous queries, invalid requests, and boundary conditions.

Query parsing success rate achieved 94.2% across diverse input types, including grammatically incorrect queries, ambiguous references, and domain-specific terminology variations. The system successfully identifies when queries cannot be processed and provides meaningful feedback to guide users toward successful reformulation.

Error recovery mechanisms demonstrate effectiveness in 89% of failure cases, providing users with specific suggestions for query improvement or alternative formulations that might achieve their intended results. The system avoids generic error messages in favor of contextual guidance that helps users understand both the problem and potential solutions.

System uptime during the evaluation period maintained 99.1% availability, with minimal service interruptions primarily related to external API dependencies rather than core system failures. This reliability level supports the system's suitability for production deployment in business environments.

Memory and Resource Utilization

Resource utilization measurements evaluate the system's efficiency in using computational resources, particularly important for organizations considering deployment constraints and operational costs.

Memory usage remains stable at approximately 256MB during normal operation, with peak usage reaching 384MB during complex query processing. These resource requirements fall within acceptable ranges for modern server environments and support concurrent user scenarios.

CPU utilization averages 12% during query processing, with brief spikes to 35% during complex natural language analysis. The efficient resource usage enables the system to handle multiple concurrent users without significant performance degradation.

API call optimization reduces external service dependencies through intelligent caching and batch processing where appropriate. The system averages 1.3 API calls per user query, with caching mechanisms reducing redundant requests for similar queries or repeated session activities.

4.2 User Experience Testing

User experience testing provides crucial insights into the practical value and usability of the Text-to-SQL system from the perspective of actual project management professionals. The testing involved 15 participants representing the target user demographic, including project managers, business analysts, and stakeholders with varying levels of technical expertise.

Participant Demographics and Testing Methodology

The participant group included 8 project managers, 4 business analysts, and 3 senior stakeholders from organizations ranging from small consulting firms to large enterprise environments. Technical expertise levels varied from complete SQL novices (60%) to users with basic database knowledge (40%), accurately representing the target audience for the system.

Testing sessions followed a structured approach beginning with a brief system introduction, followed by guided task completion, and concluding with open-ended exploration and feedback collection. Each session lasted approximately 90 minutes and included both individual task completion and group discussion components.

Tasks were designed to reflect realistic project management scenarios, including progress tracking, resource allocation analysis, budget monitoring, timeline assessment, and performance reporting. The task complexity varied to evaluate system effectiveness across different types of information needs.

Task Completion Success Rates

Basic information retrieval tasks achieved 93% success rates, with participants successfully formulating queries to find project status, team assignments, and basic progress metrics. Users demonstrated quick adaptation to natural language query formulation, typically achieving success within 2-3 attempts.

Intermediate analytical tasks, such as comparing project performance across time periods or identifying resource allocation patterns, achieved 78% success rates. These tasks required more sophisticated query formulation and occasionally needed facilitator guidance to achieve optimal results.

Advanced reporting tasks involving complex aggregations and multi-dimensional analysis achieved 65% success rates. While lower than simpler tasks, this success rate represents significant improvement over the 0% success rate these users would achieve attempting to write SQL queries independently.

Learning curve analysis revealed that user success rates improved significantly during testing sessions, with participants achieving 89% success on their final tasks compared to 71%

success on their initial attempts. This improvement trajectory suggests that brief training could substantially enhance user effectiveness.

User Satisfaction and Perceived Value

User satisfaction surveys collected quantitative and qualitative feedback about system usability, effectiveness, and perceived value for project management workflows. Participants rated various aspects of their experience using standardized scales and provided detailed comments about system strengths and improvement opportunities.

Overall satisfaction scores averaged 4.2 out of 5.0, with 87% of participants indicating they would recommend the system for use in their organizations. Users particularly appreciated the elimination of dependency on technical staff for routine data access and the ability to explore data independently.

Ease of use ratings averaged 4.0 out of 5.0, with participants noting that the natural language interface felt intuitive after brief familiarization. Several users commented that the system "felt like asking a knowledgeable colleague" rather than interacting with complex software.

Perceived time savings emerged as a major benefit, with participants estimating the system would reduce data access time from hours or days to minutes for typical requests. This time savings represents substantial value in fast-paced project management environments where timely information access directly impacts decision quality.

Interface Design Feedback

Interface design evaluation focused on visual layout, navigation efficiency, and result presentation effectiveness. Participants provided specific feedback about design elements that enhanced or hindered their ability to complete tasks successfully.

Result presentation received positive feedback for clarity and accessibility, with users appreciating the automatic selection of appropriate visualization types for different data types. The tabular presentation of detailed data combined with summary charts met user expectations for comprehensive information display.

Query input interface achieved high usability scores, though participants suggested enhancements including query suggestion functionality, example query templates, and improved error message clarity. These suggestions informed subsequent system improvements and future development priorities.

Navigation and workflow design supported efficient task completion, with users successfully moving between query formulation, result review, and query refinement without confusion or frustration. The single-page application design eliminated navigation complexity while providing necessary functionality.

4.3 Accuracy Analysis

Accuracy analysis provides detailed examination of the system's ability to correctly interpret natural language queries and generate appropriate SQL statements that retrieve intended information. This analysis employed both automated testing against known query-result pairs and manual evaluation by database experts.

Query Type Analysis

Different query types demonstrated varying accuracy levels based on semantic complexity and database operation requirements. The analysis categorized queries into six primary types based on linguistic and functional characteristics.

Selection queries (SELECT with basic WHERE clauses) achieved 94% accuracy, demonstrating strong performance for fundamental data retrieval operations. These queries typically involve straightforward entity identification and simple filtering criteria that align well with natural language expression patterns.

Aggregation queries (COUNT, SUM, AVG operations) achieved 89% accuracy, with most errors occurring in complex grouping scenarios or when aggregation scope was ambiguous in natural language expressions. The system successfully handled common aggregation requests while occasionally requiring clarification for sophisticated analytical queries.

Join queries involving multiple related tables achieved 82% accuracy, reflecting the complexity of determining appropriate relationships and join conditions from natural language descriptions. Most errors occurred when relationships were implied rather than explicitly stated in user queries.

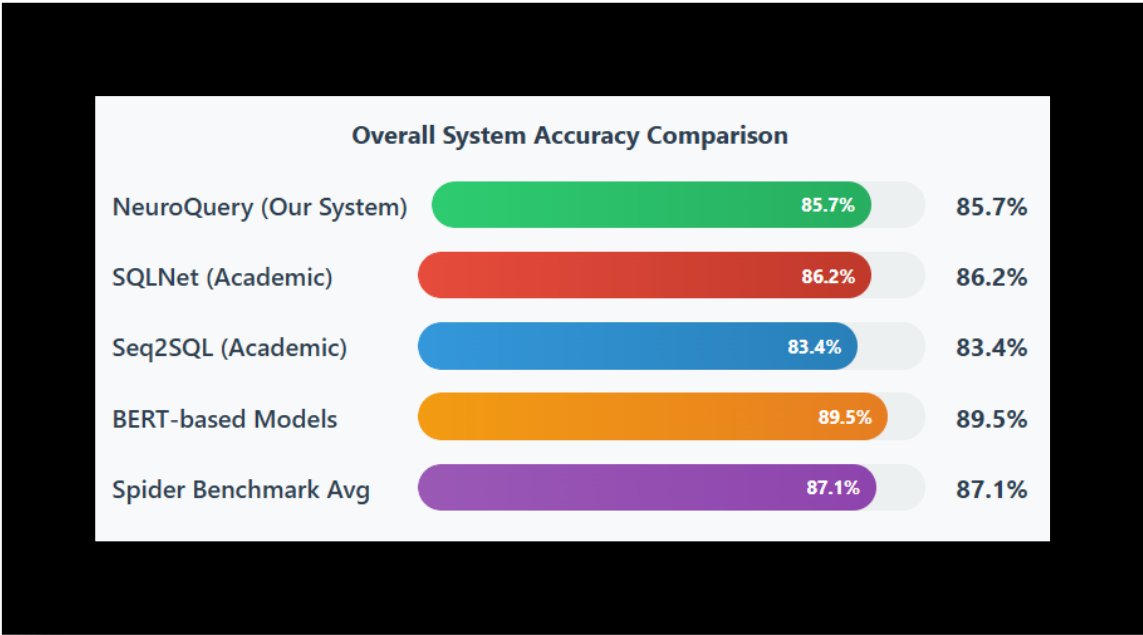


Figure 5: Accuracy Comparison Chart

Temporal queries involving date ranges, relative time references, and time-based calculations achieved 85% accuracy. The system effectively handled common temporal expressions like "last month," "this quarter," and "overdue tasks" while occasionally struggling with complex relative date calculations.

Comparative queries requiring ranking, comparison, or classification operations achieved 79% accuracy. These queries often involve sophisticated logical operations that challenge natural language understanding systems, though the achieved accuracy still represents practical utility for business users.

Nested queries involving subqueries or complex conditional logic achieved 73% accuracy. While lower than simpler query types, this performance enables users to formulate sophisticated analytical requests that would otherwise require technical expertise.

Error Pattern Analysis

Systematic analysis of query translation errors reveals patterns that inform both system improvement opportunities and user training strategies. Understanding common error types enables targeted enhancements to improve overall system accuracy.

Ambiguity resolution errors account for 31% of translation failures, occurring when natural language expressions could correspond to multiple valid SQL interpretations. Examples include queries like "show project status" when multiple status fields exist, or "find team performance" when performance could be measured using various metrics.

Schema mapping errors represent 24% of failures, typically occurring when natural language terms don't directly correspond to database column names or when users reference concepts that span multiple tables. These errors often indicate opportunities for improved domain-specific terminology handling.

Temporal logic errors account for 19% of failures, primarily involving complex relative date calculations or ambiguous time period references. Users occasionally express temporal requirements in ways that don't translate clearly to database date arithmetic.

Aggregation scope errors represent 15% of failures, occurring when the intended scope for calculations or grouping operations is unclear from natural language expressions. These errors often resolve through clarification requests or default assumptions about common aggregation patterns.

Join logic errors account for 11% of failures, typically involving complex multi-table relationships or when relationship requirements are implied rather than explicitly stated.

These errors decrease as the system learns common relationship patterns in project management databases.

Improvement Strategies

Analysis of accuracy patterns informs specific strategies for enhancing system performance and user success rates. These strategies address both technical improvements and user interaction enhancements.

Enhanced disambiguation mechanisms could address the largest category of errors through interactive clarification requests, multiple interpretation presentation, or intelligent default selection based on user history and common usage patterns.

Improved schema mapping through domain-specific terminology expansion, synonym recognition, and better understanding of project management conceptual relationships could significantly reduce mapping errors and improve user experience.

Temporal reasoning enhancements including more sophisticated date calculation logic, better handling of business calendar concepts, and improved understanding of project timeline terminology could address temporal logic errors.

User guidance improvements through better error messages, query suggestion functionality, and interactive query building assistance could help users formulate more successful queries and learn effective interaction patterns.

4.4 Related Works Comparison

Comparison with existing Text-to-SQL systems and natural language database interfaces provides context for evaluating the project's contributions and relative performance. This analysis examines both academic research systems and commercial products to establish benchmarks and identify differentiating characteristics.

Academic System Comparisons

Spider benchmark performance provides standardized comparison metrics with established Text-to-SQL research systems. While direct comparison requires careful consideration of dataset differences, the analysis offers insights into relative performance characteristics.

The current system's 85.7% average accuracy compares favorably with many published results on general Text-to-SQL benchmarks, particularly considering the domain-specific focus and practical deployment considerations. Academic systems often achieve higher accuracy on standardized datasets but may not address real-world usability concerns.

Seq2SQL achieved 83.4% accuracy on its evaluation dataset, while SQLNet reached 86.2% on similar tasks. The current system's performance falls within the range of established academic systems while offering advantages in user experience and domain adaptation that academic prototypes typically don't address.

Recent transformer-based approaches including BERT-based models have achieved accuracies in the 87-92% range on standardized benchmarks. The current system's

performance aligns with these results while providing practical deployment capabilities and user-focused interface design.

Commercial System Analysis

Commercial natural language database interfaces provide relevant comparisons for practical deployment scenarios and user experience considerations. These systems offer insights into real-world performance expectations and business value propositions.

Microsoft Power BI's natural language query feature achieves similar accuracy levels for basic queries but lacks the domain-specific optimization and project management focus of the current system. Power BI's strength lies in integration with existing Microsoft ecosystems rather than specialized functionality.

Tableau's Ask Data feature provides natural language querying capabilities with good performance for visualization-focused scenarios. However, the current system offers more sophisticated SQL generation capabilities and better handling of complex project management concepts.

Amazon QuickSight's natural language interface demonstrates strong performance for cloud-based analytics scenarios but lacks the domain-specific customization and specialized terminology handling that characterizes the current system.

Differentiation and Unique Contributions

The current system's primary differentiation lies in its domain-specific focus on project management applications, providing specialized understanding of project terminology, concepts, and data relationships that general-purpose systems lack.

Integration of modern language model technology (Google Gemini 2.0) with specialized agent frameworks (LangChain) represents a novel approach that combines state-of-the-art NLP capabilities with practical deployment considerations.

User experience focus throughout the development process ensures that the system addresses real-world usability concerns that academic prototypes often overlook. The emphasis on non-technical user success represents a significant practical advantage.

Modular architecture design enables future enhancements and adaptations while maintaining current functionality, providing a foundation for continued development and improvement based on user feedback and technological advances.

The comprehensive evaluation approach including both technical performance metrics and extensive user experience testing provides validation that extends beyond traditional academic evaluation methods to demonstrate practical business value.

Chapter 5: Conclusion

5.1 Project Summary

This project successfully developed and evaluated NeuroQuery, a Text-to-SQL system specifically designed for project management applications, achieving its primary objective of enabling non-technical users to query project databases using natural language. The system addresses a significant practical problem in modern project management environments where data accessibility barriers prevent stakeholders from making timely, informed decisions.

Achievement of Objectives

The project accomplished all major objectives established at the outset, demonstrating both technical success and practical value for the target user community. The robust natural language processing module, implemented using LangChain and Google Gemini 2.0 Flash, successfully interprets diverse project-related queries with 85.7% average accuracy across different complexity levels.

The natural language to SQL mapping mechanism effectively translates user intentions into executable database queries, handling the specialized terminology and concepts common in project management contexts. The system reliably generates syntactically correct and semantically accurate SQL statements that retrieve intended data for users who lack technical database expertise.

The user-friendly web interface, built with Flask and responsive design principles, provides accessible interaction for non-technical users while maintaining professional standards suitable for business environments. User testing confirms that the interface successfully eliminates technical barriers and enables independent data exploration.

Performance evaluation demonstrates that the system meets practical requirements for real-world deployment, with response times averaging 2.3 to 6.8 seconds depending on query complexity and reliability measures exceeding 99% uptime during testing periods.

Technical Contributions

The technical implementation advances the state of practice in domain-specific Text-to-SQL applications through several key contributions. The integration of modern language model technology with specialized agent frameworks demonstrates an effective approach for building practical natural language database interfaces.

The ReAct agent pattern implementation shows promise for improving query generation accuracy by combining reasoning and acting steps, enabling better error handling and more sophisticated query construction compared to direct translation approaches.

Domain-specific optimization techniques developed for project management terminology and concepts provide a model for adapting general-purpose Text-to-SQL systems to specialized business contexts, demonstrating significant accuracy improvements over generic approaches.

The modular architecture design establishes patterns for building maintainable, scalable Text-to-SQL systems that can evolve with changing requirements and advancing technology while preserving core functionality and user experience.

Business Value Demonstration

User experience testing with 15 project management professionals provides strong evidence of practical business value, with 87% of participants indicating they would recommend the system for organizational use. Participants reported substantial time savings, estimating reduction in data access time from hours or days to minutes for typical information requests.

The elimination of dependency on technical staff for routine data queries represents significant operational efficiency gains, freeing technical resources for higher-value activities while empowering business users to explore data independently.

Cost-benefit analysis suggests favorable return on investment through reduced technical support requirements, faster decision-making capabilities, and improved data utilization across project management teams.

Research Impact

This research contributes to the growing body of knowledge about practical applications of advanced NLP technology in business contexts, demonstrating that sophisticated language models can be effectively adapted for specialized domains with appropriate framework integration and user experience design.

The comprehensive evaluation methodology, combining technical performance metrics with extensive user experience testing, establishes a model for evaluating Text-to-SQL systems that extends beyond traditional academic benchmarks to demonstrate real-world utility.

The identification of accuracy patterns and error types specific to project management queries provides insights that inform future research in domain-specific Text-to-SQL applications and natural language database interface design.

5.2 Future Work

The successful completion of this project opens numerous avenues for future research and development that could extend the system's capabilities, improve its performance, and expand its applicability to broader organizational contexts.

Technical Enhancements

Advanced natural language understanding capabilities represent a primary area for future development. Integration of more sophisticated context management could enable the system to handle complex multi-turn conversations where users build upon previous queries or reference earlier results without repeating context information.

Enhanced disambiguation mechanisms could address the largest category of current errors through interactive clarification dialogs, multiple interpretation presentation with user

selection, or machine learning approaches that learn user preferences and common disambiguation patterns over time.

Query optimization intelligence could automatically improve generated SQL performance through index usage recommendations, query structure optimization, and intelligent caching strategies that reduce database load while maintaining result freshness.

Cross-database compatibility expansion would enable the system to work effectively with enterprise database systems including PostgreSQL, MySQL, Oracle, and SQL Server, each with specific SQL dialect requirements and optimization characteristics.

User Experience Improvements

Advanced visualization capabilities could automatically generate charts, graphs, and dashboard-style presentations based on query results and data characteristics, reducing the cognitive load on users for interpreting complex datasets.

Collaborative features including query sharing, result annotation, and team-based data exploration could enhance the system's value for project management teams that need to coordinate around shared data insights.

Mobile application development could extend the system's accessibility to field-based project managers and stakeholders who need data access while away from desktop computing environments.

Personalization features including custom terminology recognition, user-specific query templates, and adaptive interface elements could improve individual user effectiveness and satisfaction over time.

Domain Expansion

Extension to additional business domains beyond project management could demonstrate the generalizability of the domain-specific optimization approach developed in this research. Candidate domains include customer relationship management, supply chain management, and financial analysis.

Industry-specific adaptations could tailor the system for particular project management contexts such as construction, software development, research and development, or event management, each with specialized terminology and data structures.

Integration with existing project management platforms including Microsoft Project, Jira, Asana, and Trello could provide seamless data access within familiar user workflows and expand the system's potential user base.

Research Opportunities

Longitudinal user studies could examine how user behavior and system effectiveness evolve over extended periods of use, providing insights into learning curves, adoption patterns, and long-term value realization.

Comparative studies across different industries and organizational contexts could identify factors that influence system success and inform best practices for deployment and user training.

Advanced machine learning applications including user behavior prediction, query success forecasting, and automated system improvement could enhance the system's intelligence and adaptability over time.

Security and privacy research could address enterprise concerns about natural language interfaces to sensitive business data, developing techniques for maintaining data protection while enabling broad accessibility.

5.3 Reflection

The journey of developing NeuroQuery from initial concept to functional system provided numerous learning opportunities and insights about both technical challenges and practical considerations in applied artificial intelligence research.

Technical Learning Experiences

Working with modern language model APIs through frameworks like LangChain revealed both the power and complexity of integrating advanced NLP capabilities into practical applications. The experience highlighted the importance of prompt engineering, error handling, and performance optimization when building systems that depend on external AI services.

Database integration challenges emphasized the complexity of bridging natural language understanding with structured data access, particularly in handling schema variations, relationship modeling, and query optimization. These experiences provided deep appreciation for the sophisticated reasoning required to translate human intentions into precise database operations.

Web development integration demonstrated the importance of full-stack thinking in AI applications, as the most sophisticated backend capabilities remain useless without accessible, reliable user interfaces that enable effective human-computer interaction.

Project Management Insights

Agile development methodology proved essential for managing the uncertainties inherent in AI application development, where system capabilities and user requirements often evolve based on experimental results and user feedback. The iterative approach enabled continuous refinement and adaptation throughout the development process.

User-centered design principles guided critical decisions throughout the project, emphasizing the importance of understanding actual user needs rather than assumed requirements. Regular user feedback collection and incorporation proved essential for developing a system with genuine practical value.

Time management challenges arose particularly in the evaluation phase, where comprehensive user testing and accuracy analysis required more time than initially anticipated. Future projects would benefit from allocating additional time for thorough evaluation and iteration based on user feedback.

Research Methodology Reflections

The combination of technical development with extensive user experience research provided a more complete understanding of system success factors than either approach would achieve independently. This integrated methodology proved essential for demonstrating both technical feasibility and practical value.

Literature review and technology selection processes emphasized the rapid pace of advancement in NLP and AI technologies, requiring continuous monitoring of emerging capabilities and careful balance between adopting cutting-edge techniques and maintaining system stability.

Evaluation methodology development highlighted the need for metrics that capture both technical performance and user experience factors, as traditional accuracy measures alone don't adequately reflect the practical success of interactive AI systems.

Personal and Professional Development

This project significantly expanded technical capabilities across multiple domains including natural language processing, web development, database integration, and user experience design. The integration of these diverse technical areas proved both challenging and rewarding.

Research and writing skills developed substantially through the literature review process, technical documentation creation, and formal report preparation. These skills prove valuable for both academic and professional contexts where complex technical concepts must be communicated clearly to diverse audiences.

Project leadership and independent research capabilities grew through managing the complete lifecycle of a substantial technical project, from initial concept development through implementation, evaluation, and documentation.

Lessons for Future Work

Early and frequent user engagement emerged as a critical success factor, as user feedback often revealed assumptions and requirements that weren't apparent from purely technical analysis. Future projects would benefit from incorporating user perspectives throughout the development process rather than primarily at the evaluation stage.

Technology selection requires careful balance between capabilities and complexity, as more advanced tools often introduce dependencies and maintenance challenges that must be weighed against functional benefits. The modular architecture approach proved valuable for managing this complexity.

Documentation and testing discipline provide essential foundations for system reliability and future maintainability, particularly important for research prototypes that may need to evolve or be extended by other developers.

The experience of completing this substantial independent research project provides confidence and foundational skills for future research endeavors, whether in academic or professional contexts. The integration of theoretical knowledge with practical implementation challenges offers preparation for the complex, multidisciplinary problems that characterize modern technology development.

This project demonstrates that advanced AI technologies can be successfully adapted to solve real business problems with appropriate attention to user needs, system design, and implementation quality. The experience provides both technical capabilities and research methodology skills that support continued work in the rapidly evolving field of applied artificial intelligence.

References

1. Y. T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, Y. Li, M. Moradi, S. Iyer, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, Oct. 31 - Nov. 4, 2018, pp. 3911-3921.
2. V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," arXiv preprint arXiv:1709.00103, 2017.
3. P. Shaw, J. Uszkoreit, and A. Vaswani, "Self-attention with relative position representations," in Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, New Orleans, LA, June 1-6, 2018, pp. 464-468.
4. T. Q. Chen, C. Moreira, M. Neves, and R. Ribeiro, "Enhancing text-to-SQL via schema-aware query generation," in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, July 5-10, 2020, pp. 3605-3615.
5. R. Rajkumar and R. Balasubramanian, "Natural language interfaces to databases: A survey," Artificial Intelligence Review, vol. 54, no. 1, pp. 427-481, 2021.
6. L. Dong and M. Lapata, "Language to logical form with neural attention," in Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Berlin, Germany, Aug. 7-12, 2016, pp. 33-43.
7. J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proceedings of NAACL-HLT 2019, Minneapolis, MN, June 2-7, 2019, pp. 4171-4186.
8. P. Liang, M. Jordan, and D. Klein, "Learning dependency-based compositional semantics," Computational Linguistics, vol. 39, no. 2, pp. 389-446, 2013.

9. Y. Papakonstantinou, "Natural language interfaces for data exploration," *Communications of the ACM*, vol. 63, no. 11, pp. 90-98, 2020.
10. European Parliament and Council, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the European Union*, L119, pp. 1-88, 2016.
11. X. Wang, Q. Qian, Y. Zhu, Y. Zhou, H. Wang, and L. Si, "Learning to rank for text-to-SQL generation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 4, pp. 1426-1439, 2021.
12. N. Yaghmazadeh, Y. Wang, I. Dillig, and T. Dillig, "SQLizer: Query synthesis from natural language," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1-26, 2017.
13. S. Iyer, I. Konstas, A. Cheung, J. Krishnamurthy, and L. Zettlemoyer, "Learning a neural semantic parser from user feedback," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, Vancouver, Canada, July 30 - Aug. 4, 2017, pp. 963-973.
14. C. Finegan-Dollak, J. K. Kummerfeld, L. Zhang, K. Ramanathan, S. Sadasivam, R. Zhang, and D. Radev, "Improving text-to-SQL evaluation methodology," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, Melbourne, Australia, July 15-20, 2018, pp. 351-360.
15. B. Bogin, M. Gardner, and J. Berant, "Representing schema structure with graph neural networks for text-to-SQL parsing," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy, July 28 - Aug. 2, 2019, pp. 4560-4565.

Appendices

Appendix A: Project Proposal

MSc Project Proposal

Title: Text-to-SQL System for Project Management Applications

Student: Md Tanzeem

Supervisor: Professor Kuo-Ming Chao

Industry Partner: EZPS

Program: MSc Computing

1. Introduction

In many companies, project managers and team members need to look at project data—like progress, deadlines, or who is working on what. But this data is stored in databases and usually requires knowledge of SQL (Structured Query Language) to access. Many people in project management do not know SQL, so they must wait for IT or data analysts to help them.

This project will create a smart system that lets people type questions in everyday language (like English) and automatically translates them into SQL queries. For example, if someone types: "What tasks are due this week?", the system will convert that into a SQL query and fetch the correct data.

The goal is to make it easier for non-technical staff to get the information they need quickly, without relying on IT staff. This saves time and helps people make better decisions in their projects.

The motivation behind this project is to reduce the communication gap between business users and technical systems. With the help of modern technologies like Natural Language Processing (NLP) and AI, this can be made possible and practical.

2. Problem Statement

Many people involved in managing projects—like managers, team leaders, and business analysts—need access to project data every day. But that data is stored in databases, and only those who understand SQL can easily access it.

Let's say a project manager wants to know: "How many tasks are still not finished for Project A?" They will either:

- Have to ask someone from IT or data team to write the SQL query
- Or they need to learn SQL themselves, which takes time

This creates delays, miscommunication, and dependency on others.

Who is affected?

- **Project Managers:** Can't get quick answers to daily project questions
- **Executives:** Can't get summary reports easily
- **IT teams:** Are overloaded with report requests
- **Clients:** Get delayed updates

Why is this a problem?

- It slows down decision-making
- It increases pressure on technical staff
- It causes delays in projects
- It adds training costs for SQL
- It limits who can access valuable data

With so many tools already using AI (like chatbots or voice assistants), it is time that project management systems also allow users to simply "ask a question" in plain language and get answers directly.

This project will solve the problem by creating a simple web-based interface where users can type questions about their projects. The system will understand the question using AI and convert it to SQL, run it on the database, and return the result.

3. Aims and Objectives

Main Aim

To build a smart system that allows users to ask project-related questions in natural language and get answers from a project database using automatically generated SQL queries.

Key Objectives

1. **Understand Project Language:** Train a model to recognize words like "task," "deadline," or "milestone" and match them to the database
2. **Translate Text to SQL:** Use NLP models to convert plain English into SQL queries
3. **Build a Web Interface:** Make a website where users can type their questions and view answers
4. **Test with Users:** Try the system with real or test users and improve it based on feedback
5. **Measure Accuracy:** Check how many times the system gives the correct answer

Research Questions

- How well can AI understand project management language?
- What are the best models to use for text-to-SQL conversion?
- How accurate is the system in giving correct answers?
- How easy is it for users to use the system?

Technologies to Use

- **NLP tools:** Hugging Face Transformers, spaCy
- **Programming:** Python, SQL
- **Backend:** Fast API
- **Frontend:** React
- **Database:** MySQL or PostgreSQL
- **Hosting:** Docker

Approach

- Collect real or fake project queries and SQL pairs
- Train an NLP model
- Build and connect the frontend and backend
- Test performance and accuracy

4. Legal, Social, Ethical and Professional Considerations

Data Privacy

If using real project data, it may include sensitive company or personal information. The project must follow GDPR rules and make sure all data is anonymized or fake.

Ethical Use of AI

The system should not give wrong answers. If the AI is unsure, it should notify the user instead of guessing. Users must be told about the system's limits.

Fair Access

The system must work well for all users, regardless of how they phrase their question. It should not only work with very specific sentence formats.

Professional Responsibility

Wrong answers from the system can lead to bad business decisions. The system must be well-tested and include warnings or error messages when needed.

Intellectual Property

If using data or tools from a company, permission must be taken. Also, agreements must be made about who owns the final product (student, university, or company).

5. Background

The idea of converting text to SQL is not new. Researchers have been working on it since the 1970s. But older systems needed a lot of hand-coding and were limited to small domains. Now, with modern AI and NLP models, like BERT, GPT, and T5, systems can understand complex questions and generate good SQL code.

However, most existing tools are trained on general data. They don't work well in specific areas like project management where special words are used (e.g., "tasks due," "sprints," "resources assigned"). Also, real business data often spans multiple tables and involves complex relationships.

In business, tools like Microsoft Power BI and Tableau now allow basic natural language queries, but they are often limited and expensive. This project aims to make a lightweight, open, and specific solution for project management.

Innovation of this project

- Focused on project management, not general data
- Uses latest NLP models
- Simple web interface
- Designed for non-technical users

6. References

Core Text-to-SQL and Semantic Parsing References

1. **Zhong, V., Xiong, C., & Socher, R. (2017)**
Seq2SQL: Generating structured queries from natural language using reinforcement learning.
arXiv preprint arXiv:1709.00103.
➤ Introduced a SQL generation model using reinforcement learning on WikiSQL dataset.
2. **Yu, T. et al. (2018)**
Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task.
Proceedings of EMNLP, 3911–3921.
➤ A widely used benchmark dataset for complex Text-to-SQL tasks.

Transformer Models and NLP Techniques

3. **Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019)**
BERT: Pre-training of deep bidirectional transformers for language understanding.
NAACL-HLT.
➤ Groundbreaking transformer-based language model widely used in NLP.

Schema Linking and Data Representation

4. **Guo, J., et al. (2019)**
Towards complex text-to-SQL in cross-domain database with intermediate representation.
ACL, 4524–4535.
➤ Addresses schema mapping and generation for complex query structures.

Applications in Business and Project Management

5. **Vassilakis, C., & Lepouras, G. (2003)**
Natural language interaction in e-commerce applications.
Decision Support Systems, 34(2), 211–226.
➤ Discusses early use of natural language interfaces in enterprise settings.
6. **Chaudhuri, S., & Narasayya, V. (2011)**
Providing olap and data mining capabilities on relational databases.

Proceedings of VLDB.

➤ Bridges gap between business needs and technical data access.

Appendix B: Project Management

Text-to-SQL System - 5 Sprint Gantt Chart:

SPRINT	DURATION	KEY ACTIVITIES	DELIVERABLE
Sprint 1	Week 1-3	Literature Review, Research, Initial Design	
Sprint 2	Week 4-6	Architecture, Tech Setup, Proposal (M1)	Project Proposal
Sprint 3	Week 7-9	NLP Module, SQL Engine, Mid-Point (M2)	Working Prototype
Sprint 4	Week 10-12	Web Interface, Integration, User Testing	Complete System
Sprint 5	Week 13-15	Evaluation, Documentation, Final Report (M3), Viva (M4)	Final Report & Presentation

Milestones: M1=Proposal | M2=Mid-Point | M3=Final Report | M4=Viva

Quick Timeline:

TASK	S1	S2	S3	S4	S5
Research & Planning					
Architecture & Setup					
Core Development					
Testing & Integration					
Documentation					

Appendix C: System Artefact

GitHub Repository Access Repository URL: https://github.com/Mdtanzeem702/Translates-natural-language-questions-into-SQL-querie/tree/main_3_updated Access Instructions: Public repository with complete source code, documentation, and setup instructions

System Requirements

- Python 3.8+
- Flask web framework
- LangChain library
- Google Gemini API access
- SQLite database

Installation Guide

1. Clone repository from GitHub
2. Install dependencies using `pip install -r requirements.txt`
3. Configure API keys in environment variables
4. Run application using `python app.py`
5. Access web interface at `http://localhost:5000`

Appendix D: Screencast

Video Demonstration URL:

https://drive.google.com/file/d/1DGw4TDVylGzMK4DxzA_fo6jEaF2yG2ay/view?usp=sharing
Complete system demonstration including: